Algortym sortowania danych

Oskar Sałata

Kwiecień 2026

Algorytmy i struktury danych,
Politechnika Warszawska

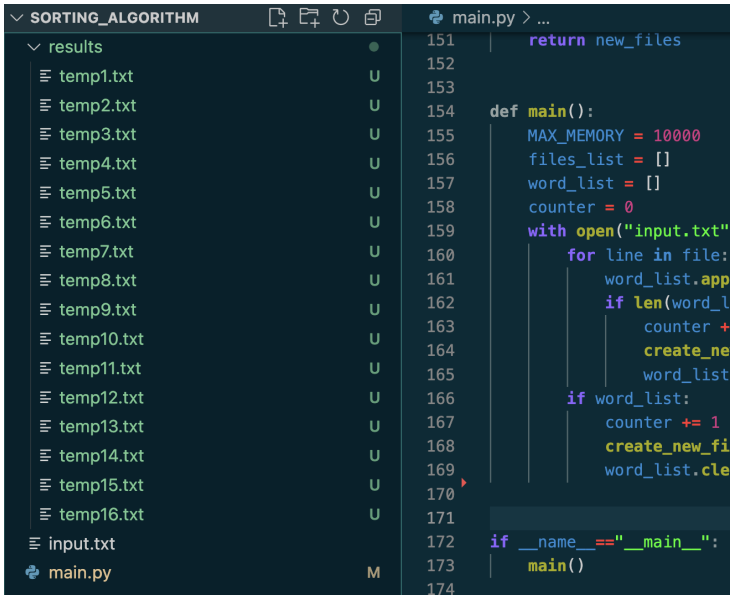
1 Wprowadzenie

Zadanym ćwiczeniem było wczytanie i posortowanie słów w kolejności alfabetycznej, jednak z tym ograniczeniem, że program miał nałożony liczbowy limit słów, które program mógł w danym momencie przechować w pamięci. Do rozwiązania tego zadania został użyty algorytm mergesort o złożoności czasowej $O(n \log n)$ wymagający $O(n)$ pamięci. Program miał być napisany w języku Python. Dane wejściowe znajdowały się w pliku 'input.txt', a posortowane słowa były zapisywane w pliku 'output.txt'.

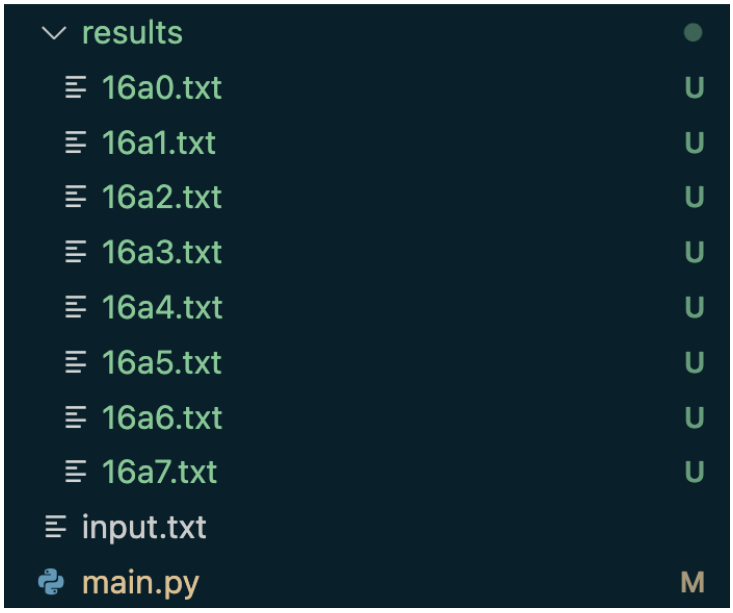
2 Rozwiązanie

Jak wspomniano wcześniej, do sortowania danych został użyty algorytm mergesort. Program otwierał plik ze słowami, a następnie wczytywał słowa do listy, aż do osiągnięcia określonego limitu pamięci (zdefiniowanego stała `MAXMEMORY`). Tak przygotowana lista była poddana sortowaniu. Aby posortować te słowa, została użyta hashmapa, która przypisywała wartości każdej literze polskiego alfabetu. W tan sposób program potrafił w krótkim czasie nadawać wagi literom, a następnie je porównywać. Zawartość posortowanej listy była zapisywana w pliku tymczasowym. Skutkiem czego program stworzył kilkanaście plików tymczasowych, w których maksymalna liczba słów wynosiła wspomnianą wcześniej liczbę `MAX MEMORY`. Następnym proces było scalanie tych plików. Następnym procesem było scalanie plików parami, odczytując po jednym słownie i porównując ich kolejność alfabetyczną. Ten proces się powtarzał, aż na końcu pozostał tylko jeden plik, który jest wynikiem całego zadania.

Dla `MAX MEMORY` = 10000 proces wygląda następująco:

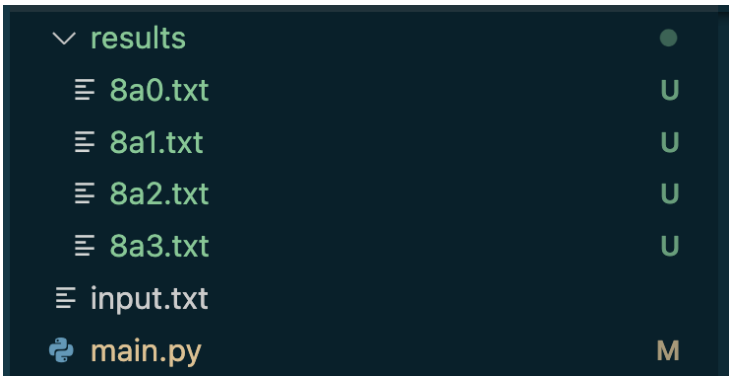


Program tworzy 16 tymczasowych plików, ponieważ skoro maksymalna pamięć RAM wynosi 10000, a słów jest 160000 to dzieląc liczbę słów na pamięć RAM wychodzi 16 plików tymczasowych.

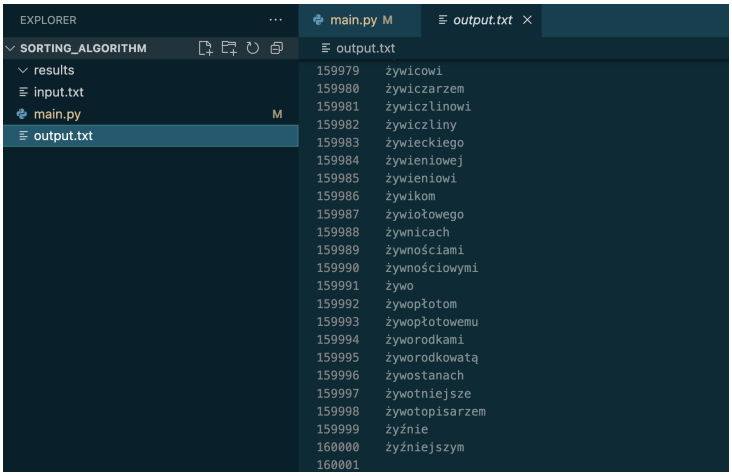


Następnie program zaczyna proces łączenia plików w pary i scalania ich.

Ten cykl trwa, dopóki nie pozostaną 2 pliki, które ostatecznie zostaną scalone w output.txt. Nazewnictwo plików powstających w kolejnych fazach polega na tym, że algorytm wykorzystuje długość listy zawierającej nazwy plików tymczasowych, a potem dla każdej iteracji scalania dodaje literę oznaczającą numer iteracji. Jest to spowodowane tym, że program na bieżąco usuwa pliki, a nie można dopuścić do sytuacji, w której program usunie plik z danymi, które nie były jeszcze ujęte w procesie scalania.

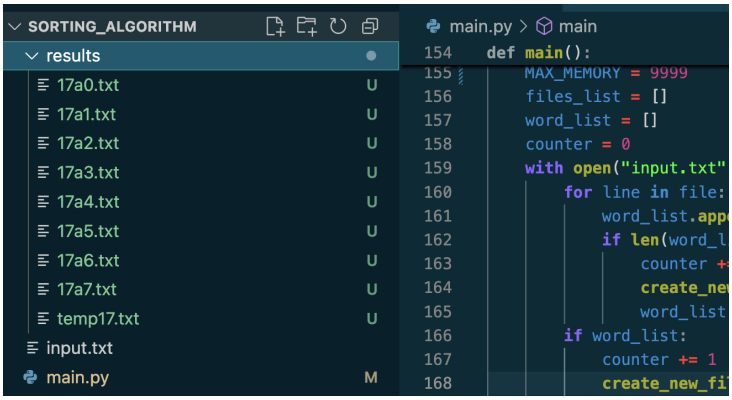


Proces przebiega poprawnie, dopóki pozostają dwa pliki, które są scalane w output.txt

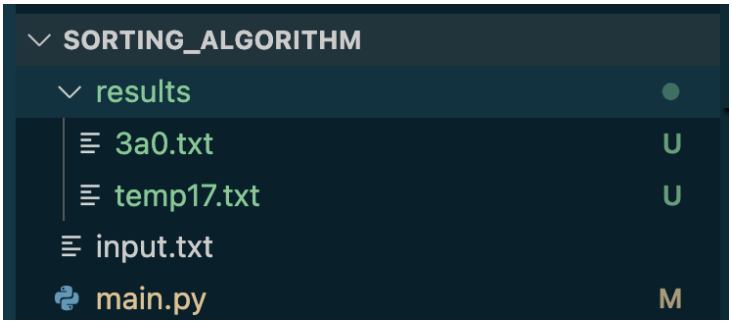


W rezultacie program poprawnie sortuje wszystkie słowa z pliku `input.txt`, nie przekraczając przy tym z góry zdefiniowanego limitu pamięci RAM przeznaczonej na przechowywanie słów.

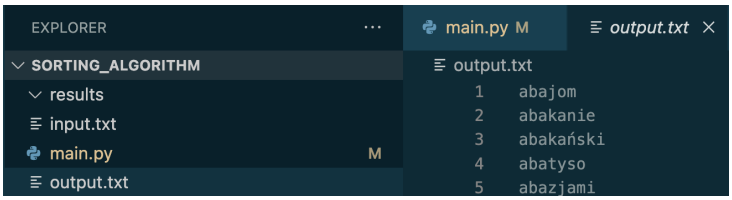
Przypadkami brzegowymi, które mogłyby spowodować niechciany wynik, były sytuacje, w których całkowita liczba słów do posortowania nie dzieliła się bez reszty przez ustalony limit pamięci RAM, lub gdy w danej fazie powstawała nieparzysta liczba plików tymczasowych. Program radzi sobie z tymi przypadkami bezbłędnie.



Program wygenerował 17 plików, z czego 16 zostało scalonych w pierwszej kolejności. Ostatni plik zostanie dołączony do procesu dopiero wtedy, gdy z pozostałych 16 plików powstanie tylko jeden.



W opisywanym scenariuszu program zredukował 16 plików do zaledwie jednego. W rezultacie na liście pozostał tylko ten nowo powstały plik oraz odłożony wcześniej plik nieparzysty. Ostatnim krokiem działania algorytmu jest scalenie tych dwóch elementów.



W końcowym etapie program scala dwa ostatnie pliki. W wyniku tej operacji wszystkie słowa zostają poprawnie posortowane i zapisane w docelowym pliku tekstowym.

3 Badania

Tabela 1: Wpływ wielkości pamięci RAM na czas wykonywania programu (sortowanie + tworzenie nowych tymczasowych plików i ich scalanie)

Wartość MAX MEMORY	Czas wykonywania programu w sekundach
1	(program nie działał)
10	5,27
50	2,23
100	1,80
1 000	1,25
10 000	1,06
100 000	0,99
160 000	0,97

Przeprowadzone badania jednoznacznie wykazują, że dostępna pamięć operacyjna ma krytyczny wpływ na czas wykonania algorytmu sortowania zewnętrznego, jednak zależność ta nie jest liniowa. W skrajnie trudnych warunkach (limit 10 słów) program wykonał się w 5,27 sekundy z powodu gigantycznego narzutu operacji wejścia/wyjścia – konieczności utworzenia, zapisania i wielokrotnego odczytania tysięcy plików tymczasowych. Zwiększenie pamięci zaledwie do 1000 elementów zredukowało czas ponad czterokrotnie (do 1,25 s). Od wartości 10 000 można zaobserwować zjawisko malejących przychodów – czas wykonania ulega spłaszczeniu, a całkowite załadowanie pliku do RAM-u (limit 160 000) przyspieszyło program już tylko o 0,09 s względem limitu 10 000. Warto również zauważyć, że przy limicie 1 elementu program nie był w stanie poprawnie zrealizować zadania, co najprawdopodobniej wynika z ograniczeń systemu operacyjnego dotyczących maksymalnej liczby jednocześnie obsługiwanych plików.

4 Wnioski

Głównym wnioskiem płynącym z realizacji zadania jest potwierdzenie wysokiej skuteczności algorytmu sortowania zewnętrznego w przetwarzaniu dużych zbiorów danych przy rygorystycznych limitach pamięci RAM. Zastosowanie struktury słownika do przypisania wag literom okazało się optymalnym rozwiązaniem problemu poprawnego sortowania polskiego alfabetu.

Zrealizowany algorytm udowadnia, że w przypadku ograniczonych zasobów sprzętowych, transfer obciążenia z pamięci operacyjnej na operacje wejścia/wyjścia dysku twardego jest opłacalnym i koniecznym kompromisem inżynierskim.